

Week 14 – Algorithm Design

■ Learning Objectives

- Differentiate brute force vs. greedy approaches.
- Recognize common patterns: max/min, accumulation, two-pointer, counting.
- Turn problem statements into step-by-step plans.

■ Key Concepts

- **Brute Force:** try all possibilities within constraints.
- **Greedy:** make locally best choice hoping for global optimum.
- **Invariants:** properties that remain true as your algorithm runs.
- **Proof by Counterexample:** test your idea on edge cases.

Example – Max Difference (Two-Pointer vs Brute Force)

```
# Brute force  $O(n^2)$ 
nums = [3, 10, 6, 1, 8]
best = 0
for i in range(len(nums)):
    for j in range(i+1, len(nums)):
        best = max(best, abs(nums[i]-nums[j]))
print(best)

# Greedy for max difference: track min/max so far  $O(n)$ 
mn = mx = nums[0]
for x in nums[1:]:
    mn = min(mn, x); mx = max(mx, x)
print(mx - mn)
```

■ Practice Problems

Practice 1 – Choose the Greedy

Given coin values [25,10,5,1], make change for N using fewest coins.

```
N = 87
coins = [25,10,5,1]
cnt = 0
for c in coins:
    take = N // c
    cnt += take
    N -= take * c
print(cnt)
```

Practice 2 – Interval Scheduling (Greedy by Finish Time)

```
intervals = [(1,4),(3,5),(0,6),(5,7),(8,9),(5,9)]
intervals.sort(key=lambda x: x[1])
ans, last_end = [], -1
for s,e in intervals:
    if s >= last_end:
        ans.append((s,e)); last_end = e
print(ans)
```

■ Strategy Checklist

- Clarify input/output and constraints.
- List patterns: counting, prefix sums, sort+scan, sets/maps, two-pointer.
- Try small cases; search for counterexamples.
- Prefer simple $O(n \log n)$ over clever $O(n^2)$ that risks TLE.

■ Homework Challenge

Given tasks with durations, schedule as many as possible in a day of length D. Print max count.

```
D = 10
tasks = [2,3,1,2,5,4]
tasks.sort()
time = taken = 0
for t in tasks:
    if time + t <= D:
        time += t; taken += 1
print(taken)
```